

基于 YOLO11 和 DeepSeek 的中药材检测与识别系统项目说明书

一、项目概述

本项目是一个面向中药材图像检测与识别场景的毕业设计系统，主要实现图片识别、批量图片识别、视频识别、摄像头识别、AI 文字说明、历史记录管理和手机端访问等功能。系统采用前后端分离架构，前端负责交互展示，Spring Boot 后端负责业务接口和数据存储，Flask 后端负责 YOLO11 模型推理和 DeepSeek 文本说明生成，Android 端通过 WebView 实现移动端访问。

项目核心思路可以概括为：用户上传图片、视频或摄像头画面后，系统将数据传递到后端，由 YOLO11 模型完成中药材目标检测，返回类别、置信度和检测框；随后结合 DeepSeek 生成文字说明，并将识别记录保存到 MySQL 数据库中，最终在网页端或手机端展示识别结果。

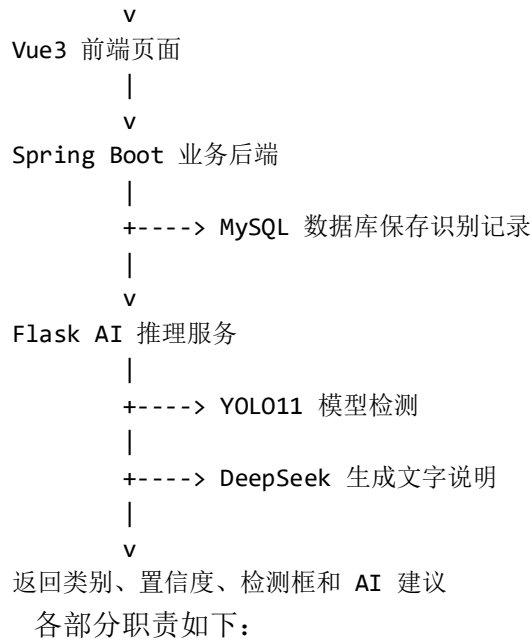
二、系统功能

功能模块	功能说明
用户管理	支持用户注册、登录和个人信息维护
单图识别	上传单张中药材图片，返回识别类别、置信度和检测框
批量识别	支持多张图片批量上传，并展示批量检测结果
视频识别	对上传视频进行逐帧检测，生成带检测框的结果视频
摄像头识别	调用摄像头画面进行实时检测，并动态显示识别结果
AI 文字说明	结合 DeepSeek 根据识别结果生成自然语言说明
历史记录	保存图片、视频和摄像头识别记录，支持查询与删除
手机端访问	Android WebView 加载网页系统，实现移动端访问

三、技术架构

系统整体采用“Vue 前端 + Spring Boot 业务后端 + Flask AI 推理后端 + MySQL 数据库 + Android WebView”的架构。

用户图片/视频/摄像头画面
|



层次	技术	主要职责
前端展示层	Vue3、Element Plus、Vite	页面展示、图片上传、结果渲染、历史记录查询
业务后端层	Spring Boot、MyBatis Plus	用户管理、文件上传、接口转发、识别记录保存
AI 推理层	Flask、Ultralytics YOLO、DeepSeek API	图片/视频/摄像头推理、AI 文本说明
数据存储层	MySQL	保存用户信息和识别历史
移动访问层	Android WebView	在手机端加载网页系统

四、项目目录结构

C:\Users\86178\Desktop\BI she	
├ MedicineDetection_vue	前端 Vue3 项目
├ MedicineDetection_springboot	Spring Boot 业务后端
├ MedicineDetection_flask	Flask AI 推理后端
├ MedicineDetection_mobile_app	Android WebView 手机端
├ yoloai.sql	MySQL 数据库初始化脚本
├ start_all.bat	Windows 一键启动入口
├ start_all.ps1	PowerShell 一键启动脚本
├ test_video.mp4	视频识别测试文件
├ 毕业论文.docx	毕业论文正文
└ 毕业设计材料_正式版	毕设正式材料目录

五、运行环境要求

建议在 Windows 环境下运行本项目，主要依赖如下：

环境	建议版本或说明
Python	Python 3.8 及以上
Node.js	Node.js 16 及以上
npm	npm 7 及以上
JDK	Java 8 或更高版本
Maven	项目自带 Maven Wrapper，可直接使用 mvnw.cmd
MySQL	MySQL 5.7 或 8.0
浏览器	Chrome、Edge 等现代浏览器

Python 主要依赖包括 flask、flask_socketio、ultralytics、openai 等；前端依赖由 MedicineDetection_vue/package.json 管理。

六、数据库配置

数据库初始化脚本位于：

C:\Users\86178\Desktop\BI she\yoloai.sql

Spring Boot 数据库配置文件位于：

C:\Users\86178\Desktop\BI
she\MedicineDetection_springboot\src\main\resources\application.properties

当前默认配置如下：

```
spring.datasource.url=jdbc:mysql://localhost:3306/yolo?serverTimezone=Asia/Shanghai
spring.datasource.username=root
spring.datasource.password=123456
server.port=9999
```

使用前需要先在 MySQL 中创建 yolo 数据库，并导入 yoloai.sql。如果本机 MySQL 用户名或密码不同，需要同步修改 application.properties。

七、一键启动方式

项目根目录中已经提供一键启动脚本：

C:\Users\86178\Desktop\BI she\start_all.bat

平时启动时，双击 start_all.bat 即可。脚本会自动检查 Python、Node.js、JDK、前端依赖和 Python 依赖，并分别启动以下服务：

服务	端口	说明
----	----	----

Flask AI 推理服务	5000	负责 YOLO11 推理和 DeepSeek 调用
Spring Boot 后端	9999	负责业务接口、数据库和转发
Vue 前端	8888	负责网页端访问

启动完成后，在浏览器中访问：

`http://localhost:8888`
如果系统部署在云服务器上，则以 `.env` 中配置的服务器地址为准。

八、手动启动方式

如果不使用一键启动脚本，也可以分别启动三个服务。

1. 启动 Flask 推理后端

```
cd "C:\Users\86178\Desktop\BI she\MedicineDetection_flask"
python main.py
```

2. 启动 Spring Boot 后端

```
cd "C:\Users\86178\Desktop\BI she\MedicineDetection_springboot"
.\mvnw.cmd -DskipTests spring-boot:run
```

3. 启动 Vue 前端

```
cd "C:\Users\86178\Desktop\BI she\MedicineDetection_vue"
npm run dev
```

九、模型文件说明

YOLO 模型权重主要位于：

```
C:\Users\86178\Desktop\BI she\MedicineDetection_flask\weights\best.pt
C:\Users\86178\Desktop\BI she\MedicineDetection_flask\weights\best.onnx
```

其中：

文件	说明
<code>best.pt</code>	PyTorch 权重文件，适合训练、验证和 Python 环境下推理
<code>best.onnx</code>	ONNX 格式模型，适合部署、跨平台推理和后续性能优化

本系统日常运行时主要使用 `best.pt` 进行推理。若后续需要提升部署性能，可以考虑使用 `best.onnx`，并结合 ONNX Runtime、GPU 或模型压缩技术进行优化。

十、核心代码位置

模块	代码位置	说明
图片识别前端页面	MedicineDetection_vue/src/views/imgPredict/index.vue	图片上传、模型选择、AI 参数和结果展示
视频识别前端页面	MedicineDetection_vue/src/views/videoPredict/index.vue	视频上传、进度接收和结果视频展示
摄像头识别页面	MedicineDetection_vue/src/views/cameraPredict/index.vue	摄像头启动、停止和实时结果展示
图片历史记录页面	MedicineDetection_vue/src/views/imgRecord/index.vue	查询和删除图片识别记录
Spring Boot 转发接口	MedicineDetection_springboot/src/main/java/com/example/medicinedetection/controller/PredictionController.java	接收前端识别请求，转发给 Flask，并保存结果
文件上传	MedicineDetection_springboot/src/main/java/com/example/medicinedetection/controller/FileController.java	处理图

接口		片、视频等文件上传
图片记录接口	MedicineDetection_springboot/src/main/java/com/example/medicinedetection/controller/ImgRecordsController.java	图片识别记录查询、保存和删除
Flask主服务	MedicineDetection_flask/main.py	AI 推理接口、视频识别、摄像头识别和DeepSeek调用逻辑
图片推理封装	MedicineDetection_flask/utils/predictImg.py	加载YOLO模型并完成图片检测
批量推理封装	MedicineDetection_flask/utils/predictBatch.py	多图片批量检测
DeepSeek调用	MedicineDetection_flask/utils/chatApi.py	调用DeepSeek接口生成文字说明
Android WebView	MedicineDetection_mobile_app/android_webview/app/src/main/java/com/medicinedetection/app/MainActivity.java	手机端加载网页系统

十一、主要业务流程

1. 图片识别流程

用户上传图片
|
Vue 前端提交图片路径、模型名称和置信度阈值
|
Spring Boot 接收请求并转发给 Flask
|
Flask 调用 YOLO11 模型进行图片检测
|
DeepSeek 根据识别结果生成文字说明
|
Spring Boot 保存识别记录到 MySQL
|
Vue 前端展示检测框、类别、置信度和 AI 建议

2. 视频识别流程

用户上传视频
|
Flask 读取视频帧
|
YOLO11 对每一帧进行目标检测
|
绘制检测框并生成结果视频
|
前端展示处理进度和结果视频

3. 手机端访问流程

用户打开 Android App
|
MainActivity 初始化 WebView
|
WebView 加载系统网页地址
|
用户在手机端使用图片识别、历史记录等功能

十二、实验结果说明

模型训练结果可在以下目录查看：

C:\Users\86178\Desktop\BI she\MedicineDetection_flask\runs\detect\train

主要实验指标如下：

指标	结果	含义
Precision	0.8853	模型预测为目标的结果中，预测正确的比例
Recall	0.8488	真实目标中被模型成功检测出来的比例
mAP@0.5	0.8951	IoU 阈值为 0.5 时的平均检测精度
mAP@0.5:0.95	0.7951	多个 IoU 阈值下综合计算的平均精度
训练轮数	200 轮	模型训练迭代次数
检测类别	45 类中药材	系统支持识别的中药材类别数量

从实验结果来看，模型在当前数据集上具备较好的检测能力，可以支撑毕业设计系统演示和原型验证。

十三、使用注意事项

- 启动系统前，需要先确认 MySQL 已启动，并且 yolo 数据库已导入。
- 如果 9999、5000 或 8888 端口被占用，需要先关闭占用端口的程序，或者修改对应配置。
- 视频识别会比图片识别耗时更长，CPU 环境下等待时间可能较明显。
- DeepSeek 文字说明依赖网络和 API 配置，如果接口不可用，图片检测功能仍可运行，但 AI 建议可能无法生成。
- 手机端 WebView 加载的是网页系统地址，如果部署地址发生变化，需要同步修改 Android 端加载地址。
- 如果模型识别效果不稳定，可以适当调整置信度阈值，或补充更多训练数据重新训练模型。

十四、常见问题

1. 前端页面打不开怎么办？

先确认 Vue 服务是否启动成功，并查看终端是否显示 `http://localhost:8888`。如果端口被占用，可以修改 `MedicineDetection_vue/.env` 中的 `VITE_PORT`。

2. 图片上传成功但识别失败怎么办？

需要检查 Spring Boot 和 Flask 是否都已启动。Spring Boot 默认端口为 9999，Flask 默认端口为 5000。同时确认模型文件 `best.pt` 是否存在。

3. 数据库连接失败怎么办？

检查 MySQL 是否启动、数据库名是否为 yolo，以及 application.properties 中的用户名和密码是否正确。

4. DeepSeek 没有返回说明怎么办？

检查网络连接、API Key 配置和 DeepSeek 接口是否可用。DeepSeek 只负责文字说明，不影响 YOLO11 的图像检测结果。

5. 视频识别速度慢怎么办？

视频识别需要对视频帧逐帧推理，CPU 环境下速度较慢是正常情况。后续可以通过 GPU 推理、ONNX Runtime、降低视频分辨率或抽帧检测来优化。

十五、项目特点

本项目不是单独的模型训练实验，而是完成了从数据输入、模型推理、结果说明、数据库保存、网页端展示到手机端访问的完整工程闭环。系统结构清晰，功能覆盖图片、批量图片、视频和摄像头等多种识别方式，具有一定的教学演示价值和工程实践意义。

十六、后续改进方向

- 扩充中药材数据集，提升复杂场景下的泛化能力。
- 优化类别均衡性，减少相似药材之间的误检。
- 接入权威中医药知识库，提高 AI 文字说明的专业性。
- 使用 GPU、ONNX Runtime 或模型压缩技术提升推理速度。
- 配置正式域名和 HTTPS，提高系统部署安全性。
- 增加管理员审核和人工纠错功能，形成更完整的数据闭环。